

PROJECT REPORT

ESP32 Industrial IoT Monitoring & Control System

IoT · Industrial Automation · Embedded Systems · Wireless Monitoring & Control

Project type	Student-built prototype
Author / Team	<i>To be updated</i>
Institution	<i>To be updated</i>
Guide	<i>To be updated</i>
Version / Date	<i>To be updated</i>

Table of Contents

1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives.....	3
4. System Overview.....	4
5. Hardware Components	4
6. Software Architecture	5
7. ESP-NOW Communication	8
8. Web Dashboard	9
9. Hardware Connections.....	11
10. System Working	12
11. Testing and Validation	12
12. Results	14
13. Limitations	14
14. Future Scope	14
15. Conclusion.....	15
16. References	15
17. Appendix – Diagram Index	15

Status labels used in this report. **Implemented:** present in the current design, as described in the project information (firmware source is not yet included in the repository). **Tested:** validated on hardware and recorded in Section 11; none recorded yet. **Planned:** near-term work not yet done (validation, documentation of Sender pins, photographs). **Future:** possible extensions, not implemented. Items that depend on the firmware source are marked *To be verified against source*.

1. Introduction

This project is an ESP32-based system for monitoring environmental and object-detection parameters, with local relay control. It uses two ESP32 boards:

- **Sender ESP32** reads a DHT11 (temperature, humidity), an MQ-2 gas sensor and two IR sensors.
- **Receiver ESP32** receives the data over ESP-NOW, shows it on a 1.3" OLED, hosts a web dashboard on its own Wi-Fi access point, and drives two relays.

The design follows the basic Industrial IoT pattern: sensors collect field data, wireless communication transfers it, and a local digital interface provides monitoring and control.

2. Problem Statement

Monitoring temperature, humidity, gas level and object presence usually needs wired sensor runs or network infrastructure such as a router or cloud service. This project builds a low-cost alternative from common hardware: a wireless sensor node feeding a local display, a web dashboard and switching outputs, with no router or internet connection needed for local monitoring.

3. Objectives

- Wireless sensor data transmission (ESP-NOW)
- Real-time temperature and humidity monitoring
- Gas-level monitoring (raw sensor level, not calibrated concentration)
- IR-based object detection/status monitoring
- Local OLED display
- Web-based monitoring from a laptop or smartphone
- Live graphs of gas and temperature trends
- Relay control from the dashboard
- Two-ESP32 distributed architecture

4. System Overview

Figure 1 - Overall System Architecture

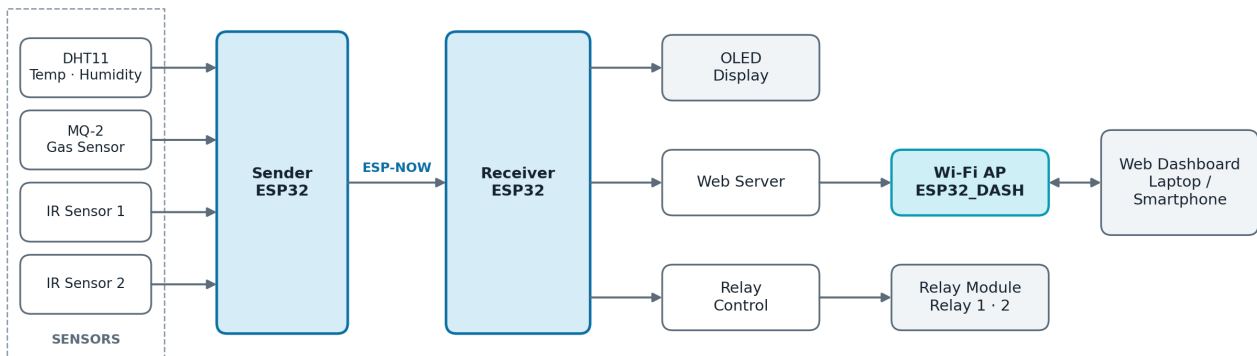


Figure 1 – Overall system architecture.

The Sender collects sensor data and transmits it over ESP-NOW. The Receiver passes it to three outputs: the OLED, the web server (reached through the ESP32 Wi-Fi access point) and the relay control.

Table 1 – Implementation status

Capability	Implemented	Tested	Planned / Future
Sensor acquisition (DHT11, MQ-2, IR 1, IR 2)	Yes	Not yet	Validation planned
ESP-NOW transmission and reception	Yes	Not yet	Validation planned
OLED display	Yes	Not yet	Validation planned
Wi-Fi AP and web dashboard with live charts	Yes	Not yet	Validation planned
Relay control from dashboard	Yes	Not yet	Validation planned
Sender GPIO documentation, photographs, screenshots	–	–	Planned
Calibrated gas measurement, cloud, authentication, alerts	No	–	Future (Section 14)

5. Hardware Components

Table 2 – Hardware components

#	Component	Qty	Role
1	ESP32 development board	2	Sender and Receiver
2	DHT11 temperature and humidity sensor	1	Temperature, humidity
3	MQ-2 gas sensor	1	Raw gas sensor level
4	IR sensor	2	Object detection/status
5	1.3" OLED display (I2C, U8g2)	1	Local display on the Receiver
6	Relay module	1	Relay 1 and Relay 2 outputs
7	Jumper wires, breadboard / prototype wiring	–	Interconnects

#	Component	Qty	Role
8	USB power / programming connections	–	Power and flashing

Relay module channel count, OLED controller variant and I2C address, sensor supply voltages and load ratings: Not specified in the current implementation.

Table 3 – Receiver pin configuration (confirmed)

Function	GPIO
OLED I2C SDA	21
OLED I2C SCL	22
Relay 1	25
Relay 2	26

Sender pins (DHT11, MQ-2, IR Sensor 1, IR Sensor 2): To be updated according to final hardware wiring.

Photographs (placeholders). Hardware setup, Sender ESP32, Receiver ESP32: *[Insert actual hardware photographs here]*

6. Software Architecture

Table 4 – Technology stack

Layer	Technology
Hardware	ESP32, DHT11, MQ-2, IR sensors, OLED, relay module
Communication	ESP-NOW, Wi-Fi Access Point
Embedded software	Arduino framework, C/C++
Libraries	WiFi.h, esp_now.h, WebServer.h, Wire.h, U8g2lib.h, esp_wifi.h
Web	HTML, CSS, JavaScript, Chart.js (live temperature and gas trend graphs)

6.1 Sender ESP32

Figure 2 - Sender ESP32 Block Diagram

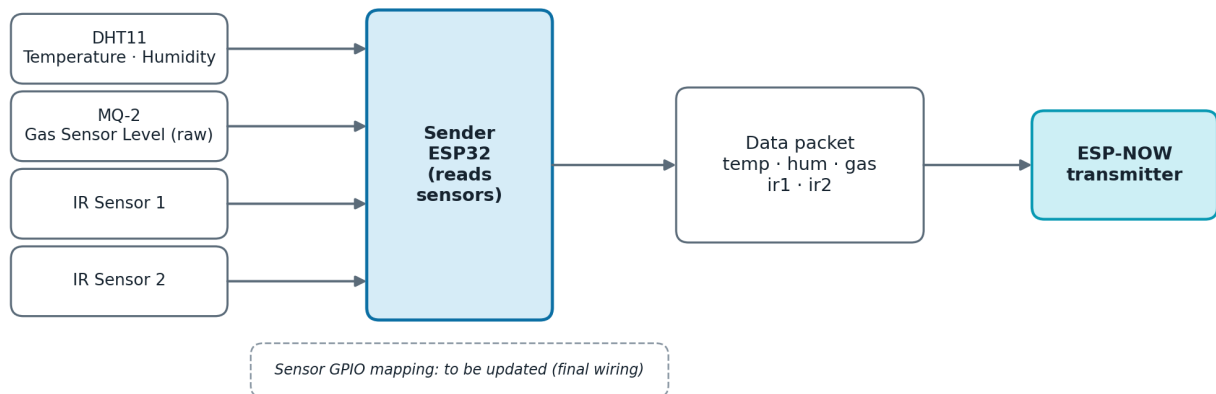


Figure 2 – Sender ESP32 block diagram.

6.2 Receiver ESP32

Figure 3 - Receiver ESP32 Block Diagram

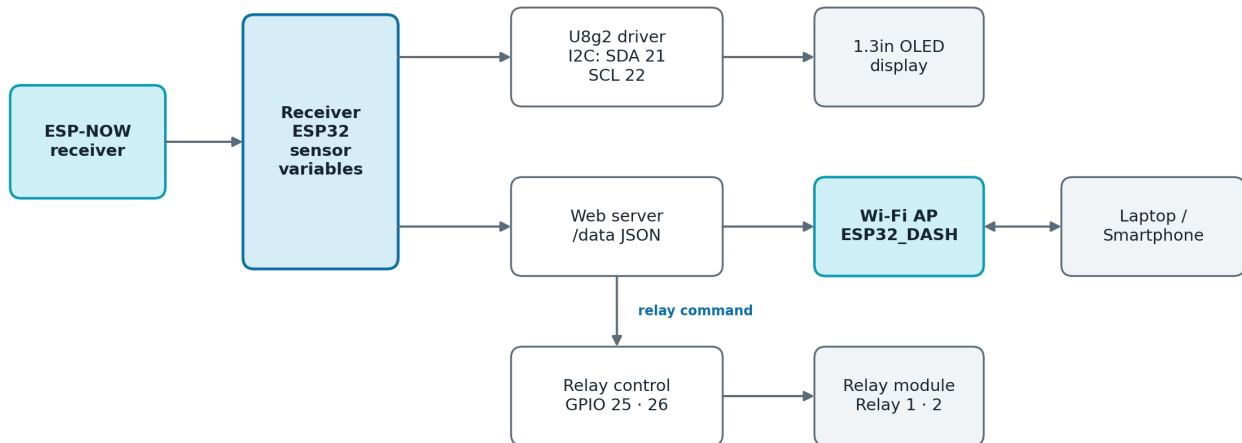


Figure 3 – Receiver ESP32 block diagram.

6.3 Firmware flow

Figure 6 – Software Flowchart

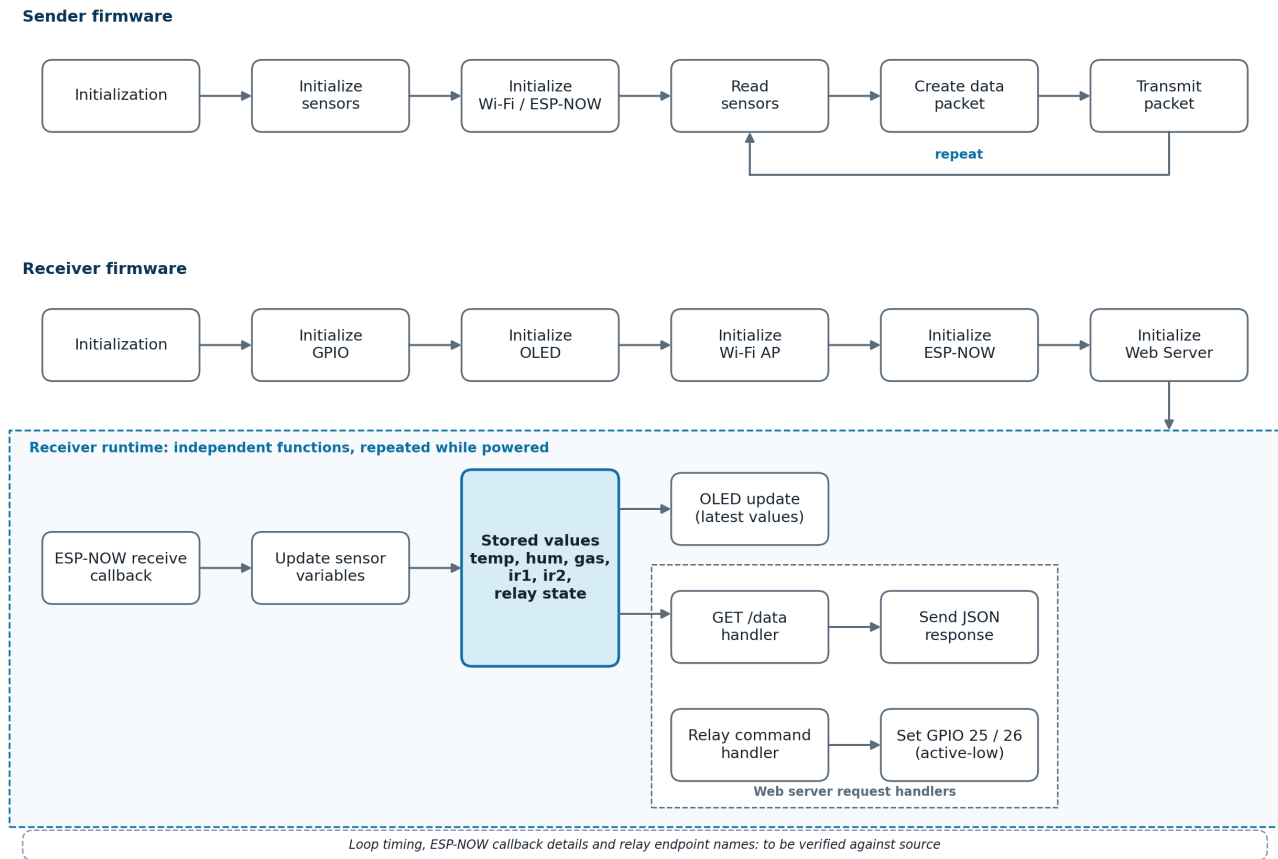


Figure 6 – Software flowchart.

The Sender initializes, then repeatedly reads sensors, builds a packet and transmits it. On the Receiver, initialization is followed by independent runtime functions rather than one serial sequence:

- The **ESP-NOW receive callback** updates the stored sensor values.
- The **OLED update** shows the latest stored values.
- The **web server** serves two separate request handlers: `GET /data` returns the stored values as JSON, and the relay command handler sets GPIO 25 / GPIO 26. Relay switching does not depend on the `/data` response.

The firmware is structured so that the web server keeps responding while sensor updates arrive.

6.4 Implementation details

Table 5 – Firmware details

Item	Current documentation	Status
Receiver GPIOs	OLED SDA 21, SCL 22; Relay 1 = 25, Relay 2 = 26	Confirmed
Sender GPIOs	–	To be updated according to final hardware wiring
Data structure	<code>temp, hum, gas, ir1, ir2</code>	Confirmed

Item	Current documentation	Status
JSON structure	temp, hum, gas, ir1, ir2, r1, r2 (approximate)	Field types to be verified
Data endpoint	/data	Confirmed
Relay endpoints	–	To be verified
Receiver Wi-Fi	Access point, SSID ESP32_DASH	Exact mode setting to be verified
Wi-Fi channel, Sender and Receiver MAC	–	To be verified
ESP-NOW receive callback	Updates stored sensor values	Details to be verified
OLED initialization	U8g2, I2C on GPIO 21/22	Constructor / address to be verified
Dashboard refresh interval	Periodic automatic refresh	Interval to be verified
Cache handling	Cache prevention used	Method to be verified
Chart.js	Gas and temperature trend charts	Point count, CDN vs. local loading to be verified
Relay logic	Active-low for this relay module	Confirmed

7. ESP-NOW Communication

ESP-NOW is an Espressif protocol that lets ESP32 devices exchange short packets directly, without a Wi-Fi router. It has low protocol overhead and suits local sensor-to-controller links. Range and reliability have **not** been measured for this project.

Figure 4 - ESP-NOW Communication Flow

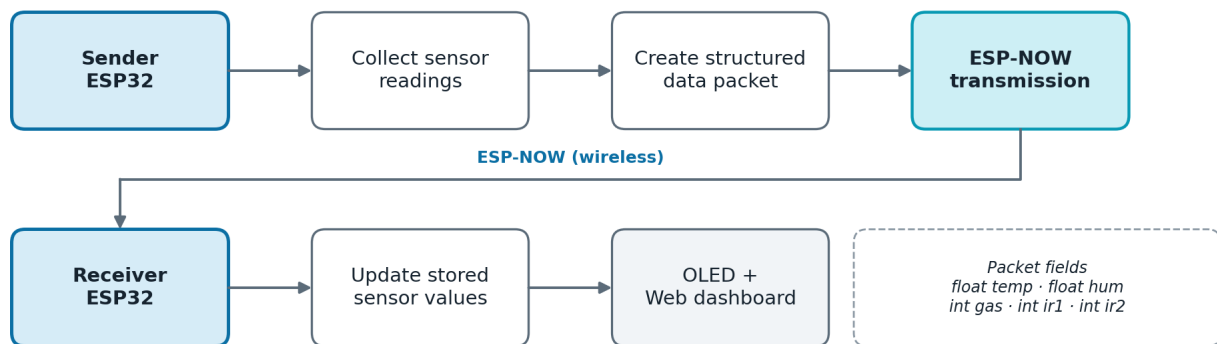


Figure 4 – ESP-NOW communication flow.

Table 6 – Data packet fields

Field	Type	Meaning
temp	float	Temperature from the DHT11
hum	float	Relative humidity from the DHT11
gas	int	Raw MQ-2 reading (Gas Sensor Level)

Field	Type	Meaning
ir1	int	IR Sensor 1 value
ir2	int	IR Sensor 2 value

Sender and Receiver must use the same Wi-Fi channel for ESP-NOW to work. *Channel selection, peer MAC handling and the meaning of each IR value: to be verified against source.*

8. Web Dashboard

The Receiver hosts a local web server (`WebServer.h`) on its own access point, so the data path works without internet access. Several devices can connect to the access point, subject to the practical limits of the ESP32; no maximum user count is claimed. *If Chart.js is loaded from a CDN, the client needs internet access for the charts; to be verified.*

Access point	Value
SSID	ESP32_DASH
Password	12345678 (defined in firmware; change before deployment)

8.1 Layout

Table 7 – Dashboard layout

Zone	Content
Top	Project title / system status
Metrics	Temperature, Humidity, Gas Sensor Level, IR Sensor 1, IR Sensor 2
Graphs	Gas Level Trend, Temperature Trend
Controls	Relay 1, Relay 2

The layout is responsive for laptop and mobile screens, with automatic refresh.

Screenshots (placeholders). Dashboard on laptop and on smartphone: *[Insert actual dashboard screenshots here]*

8.2 Data flow and API

Figure 7 – Web Dashboard Data Flow

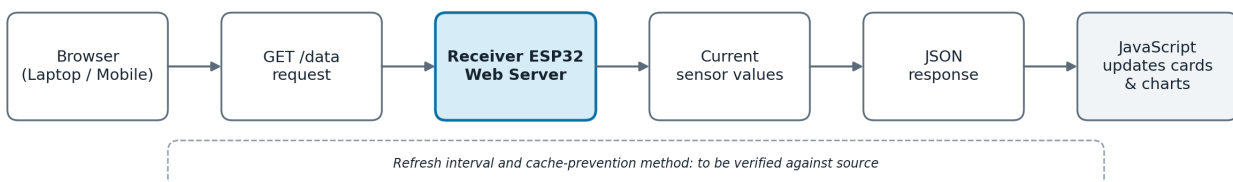


Figure 7 – Web dashboard data flow.

The browser sends `GET /data`; the Receiver returns the current values as JSON; JavaScript updates the cards and adds temperature and gas values to the charts. Cache prevention makes the browser fetch current data instead of a cached response.

Table 8 – `/data` response fields

Field	Description
temp	Temperature (DHT11)
hum	Humidity (DHT11)
gas	Raw MQ-2 Gas Sensor Level
ir1	IR Sensor 1 value
ir2	IR Sensor 2 value
r1	Relay 1 state
r2	Relay 2 state

Engineering note. The MQ-2 output shown in this project represents the sensor reading/level used for monitoring. Accurate gas concentration in PPM requires proper sensor calibration, load resistance considerations, environmental compensation, and gas-specific calibration.

8.3 Relay control

Figure 8 – Relay Control Flow

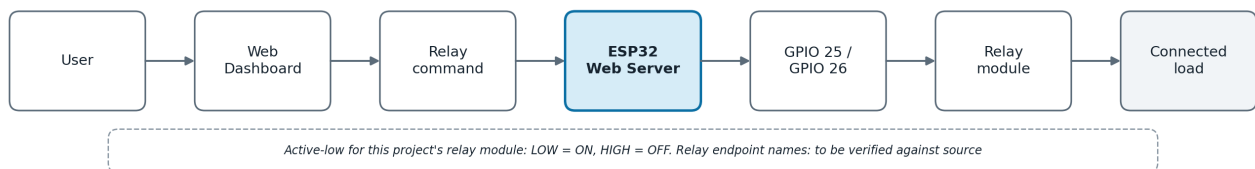


Figure 8 – Relay control flow.

Relay commands from the dashboard are handled by the Receiver web server separately from the `/data` request, and set GPIO 25 (Relay 1) and GPIO 26 (Relay 2). In the relay module used in this project the outputs are **active-low**: `LOW` = relay ON, `HIGH` = relay OFF. This applies to this project's module only. *Relay endpoint names: to be verified.*

8.4 OLED display

Figure 9 - OLED Display Data Flow

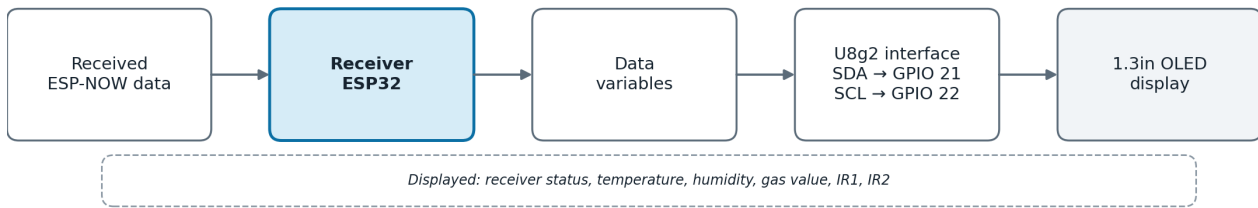


Figure 9 – OLED display data flow.

The OLED shows receiver status, temperature, humidity, gas value, IR1 and IR2, using U8g2 over I2C (SDA GPIO 21, SCL GPIO 22).

OLED photograph (placeholder). *[Insert actual OLED photograph here]*

8.5 Network notes

The access point is local; dashboard traffic stays within the ESP32 network. This is not production-grade security. Change the password before serious use and avoid hard-coding credentials in a public repository.

9. Hardware Connections

Figure 5 - Hardware Connection Diagram

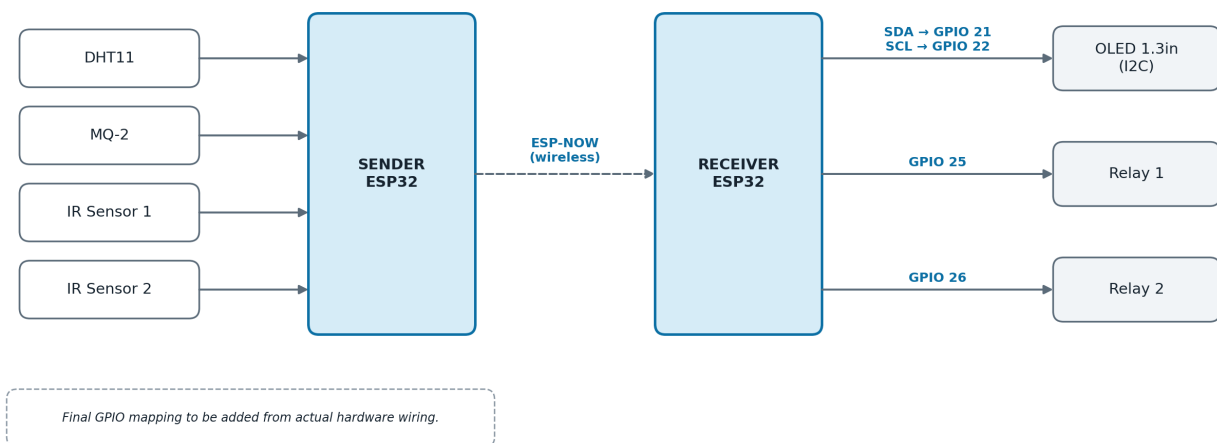


Figure 5 – Hardware connection diagram.

Sender pin assignments: **final GPIO mapping to be added from actual hardware wiring.** Power arrangement beyond USB and relay load wiring: Not specified in the current implementation.

10. System Working

1. The Sender ESP32 powers up.
2. Sensors are initialized.
3. The Sender reads temperature and humidity from the DHT11.
4. The Sender reads the MQ-2 gas sensor.
5. The Sender reads IR Sensor 1 and IR Sensor 2.
6. Readings are stored in a structured data packet.
7. The packet is transmitted using ESP-NOW.
8. The Receiver ESP32 receives the packet.
9. The Receiver updates its sensor variables.
10. The latest values are shown on the OLED.
11. The Receiver web server provides sensor data at `/data`.
12. A laptop or phone browser requests the latest data.
13. JavaScript updates the metric cards.
14. Temperature and gas values are added to the live charts.
15. The user controls Relay 1 and Relay 2 from the dashboard; the relay command is handled separately from data requests.

Figure 10 - Complete End-to-End System Flow

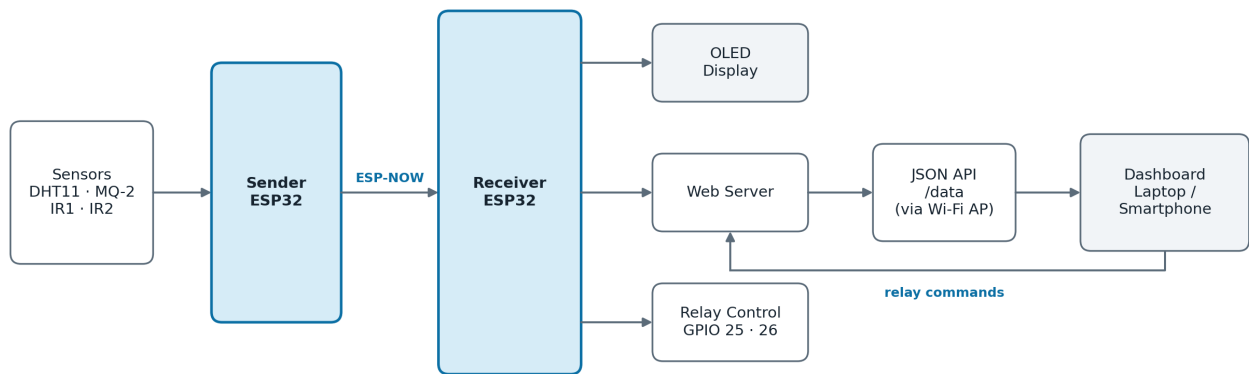


Figure 10 – Complete end-to-end system flow.

The Sender reads periodically, the Receiver updates when packets arrive, the dashboard polls `/data`, and the OLED shows the latest received values. *Intervals: to be verified against source.*

11. Testing and Validation

No tests have been performed or recorded. Every entry below is **Pending Validation**; actual results and PASS/FAIL status will be entered only after hardware testing.

Table 9 – Test matrix

ID	Test Description	Expected Result	Actual Result	Status	Remarks
T01	Sender ESP32 startup	Boots; sensors and ESP-NOW initialize	To be filled	Pending Validation	

ID	Test Description	Expected Result	Actual Result	Status	Remarks
T02	DHT11 temperature reading	Plausible temperature reported	To be filled	Pending Validation	
T03	DHT11 humidity reading	Plausible humidity reported	To be filled	Pending Validation	
T04	MQ-2 sensor reading	Gas Sensor Level changes with gas exposure	To be filled	Pending Validation	Warm-up needed; uncalibrated
T05	IR Sensor 1	Value changes on object presence	To be filled	Pending Validation	
T06	IR Sensor 2	Value changes on object presence	To be filled	Pending Validation	
T07	ESP-NOW communication	Receiver gets packets from Sender	To be filled	Pending Validation	
T08	Receiver OLED display	OLED shows latest values	To be filled	Pending Validation	
T09	Web dashboard access	Dashboard loads via <code>ESP32_DASH</code>	To be filled	Pending Validation	
T10	Live dashboard update	Cards refresh automatically	To be filled	Pending Validation	
T11	Gas graph update	Gas trend chart updates	To be filled	Pending Validation	
T12	Temperature graph update	Temperature trend chart updates	To be filled	Pending Validation	
T13	Relay 1 control	Relay 1 toggles from dashboard	To be filled	Pending Validation	Active-low
T14	Relay 2 control	Relay 2 toggles from dashboard	To be filled	Pending Validation	Active-low
T15	Mobile dashboard access	Dashboard usable on smartphone	To be filled	Pending Validation	

Table 10 – Troubleshooting

Problem	Possible causes
ESP-NOW data not received	Different Wi-Fi channel; incorrect MAC address; ESP-NOW initialization failure; sender/receiver not configured correctly
Dashboard shows --	Browser cannot reach <code>/data</code> ; Receiver not receiving ESP-NOW data; web server issue; browser cache; network connection issue
OLED not displaying	Incorrect I2C wiring; wrong OLED address; incorrect SDA/SCL; library configuration
Relay not switching	Incorrect GPIO; active-low logic misunderstanding; relay power issue; wiring problem
Gas value unstable	MQ-2 warm-up; environmental changes; sensor characteristics; lack of calibration

12. Results

No numerical results are reported. The prototype is designed to demonstrate the capabilities below; evidence will be added after validation.

Capability	Evidence
Sensor data acquisition	[Add Serial Monitor screenshot]
Wireless transmission over ESP-NOW	[Add Serial Monitor screenshot]
Local visualization on the OLED	[Add OLED photograph / capture]
Web dashboard with live charts	[Add dashboard screenshots]
Relay control	[Add photograph or video]
Laptop and mobile access	[Add screenshots]

13. Limitations

1. MQ-2 values are raw sensor levels, not calibrated PPM measurements, unless calibration is performed.
2. ESP-NOW range depends on environment, antenna, interference and hardware; it has not been measured.
3. This is a prototype and is not certified for industrial safety applications.
4. Relay outputs must not be connected to dangerous loads without appropriate electrical protection and isolation.
5. The Wi-Fi password is defined in firmware and should be changed for deployment.
6. Sensor accuracy depends on sensor quality, calibration, environment and wiring.
7. No database or cloud storage is used.
8. The project focuses on local monitoring and control.

14. Future Scope

None of the following is implemented.

Area	Possible improvements
Connectivity	MQTT, cloud IoT dashboard, remote monitoring, multiple sensor nodes
Data	Database logging, historical storage, data export, SD card logging
Security	User authentication, HTTPS, better network security
Sensing	Sensor calibration, industrial-grade sensors, additional sensors
Automation	Alarm/buzzer, automatic relay rules, emergency shutdown system
Alerts and apps	Email/Telegram alerts, mobile application
Reliability	OTA firmware updates, watchdog and fault recovery
Hardware	Enclosure, PCB, industrial power supply

15. Conclusion

The project demonstrates a local ESP32-based IoT architecture combining sensor acquisition, ESP-NOW wireless communication, local processing, OLED visualization, web-based monitoring and relay control. It is a prototype: it makes no claim of industrial certification, calibrated gas measurement or production readiness. It provides a base that can be extended toward larger IoT and automation applications through the items in Section 14.

16. References

1. Espressif Systems, *ESP32 Series Datasheet and Technical Reference Manual*. <https://www.espressif.com/en/support/documents/technical-documents>
2. Espressif Systems, *ESP-NOW, ESP-IDF Programming Guide*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
3. Espressif Systems, *Arduino core for the ESP32 documentation*. <https://docs.espressif.com/projects/arduino-esp32/>
4. olikraus, *U8g2 library documentation*. <https://github.com/olikraus/u8g2/wiki>
5. Aosong, *DHT11 datasheet* (version to be updated).
6. Hanwei Electronics, *MQ-2 gas sensor datasheet* (version to be updated).
7. Chart.js Contributors, *Chart.js documentation*. <https://www.chartjs.org/docs/>

17. Appendix – Diagram Index

All ten diagrams are embedded in this document. The image files are also stored separately in the `Documentation/` folder of the repository.

Table 11 – Diagram index

Figure	Diagram	Section	Repository file
1	Overall system architecture	4	<code>Documentation/System_Block_Diagram.png</code>
2	Sender ESP32 block diagram	6.1	<code>Documentation/Sender_Block_Diagram.png</code>
3	Receiver ESP32 block diagram	6.2	<code>Documentation/Receiver_Block_Diagram.png</code>
4	ESP-NOW communication flow	7	<code>Documentation/ESP_NOW_Communication_Flow.png</code>
5	Hardware connection diagram	9	<code>Documentation/Circuit_Diagram.png</code>
6	Software flowchart	6.3	<code>Documentation/Software_Flowchart.png</code>
7	Web dashboard data flow	8.2	<code>Documentation/Dashboard_Data_Flow.png</code>
8	Relay control flow	8.3	<code>Documentation/Relay_Control_Flow.png</code>
9	OLED display data flow	8.4	<code>Documentation/OLED_Data_Flow.png</code>
10	Complete end-to-end system flow	10	<code>Documentation/End_to_End_Flow.png</code>